multiCooperativa

App móvil multi-cooperativa con asistente de IA

Trabajo Fin de Máster · Máster de Desarrollo con IA (BIG school)

Juan Chaparro García · Octubre 2026

El problema: un socio poco informado

Convocatorias en papel

Cartas y tablón de anuncios: el socio se entera tarde o no se entera.

Documentos fuera de alcance

Estatutos, reglamentos y actas no están a mano.

Descuadres detectados tarde

Albaranes y facturas en papel: impagos y cobros pendientes se ven tarde o nunca.

Objetivo: conectar a cada cooperativa con sus socios y controlar albaranes y facturas con un asistente de IA que respeta los permisos

La solución y sus cuatro perfiles

Una app estándar para cualquier cooperativa; un usuario, varias cooperativas, datos aislados.

El backend decide; la app solo oculta.

Socio

Informarse, votar, leer documentos y ver su capital

Consejo rector

Comunicados, asambleas, vigilar pagos y cobros

Gestor

Registrar y conciliar albaranes y facturas

Administrador

Dar acceso con invitaciones, asignar perfiles y marca

Módulos del MVP: hecho y pendiente

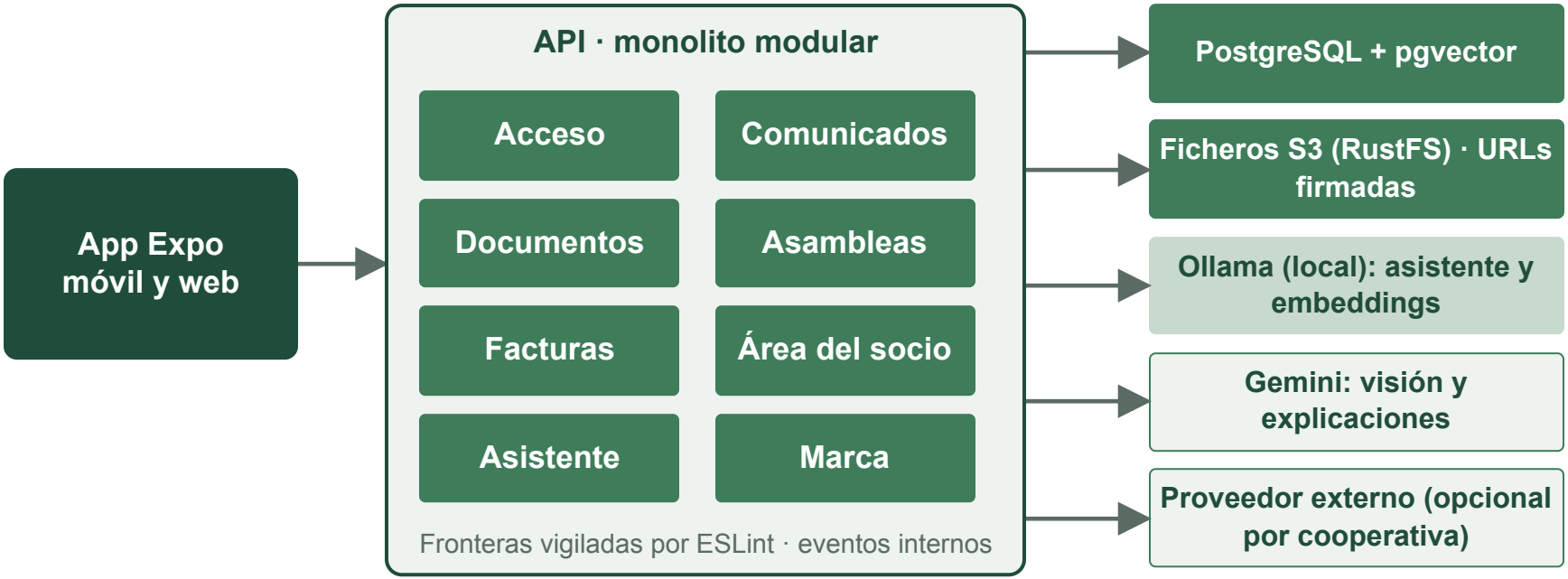
Implementado: los 7 módulos

- Acceso y perfiles, miembros e invitaciones por enlace
- Comunicados
- Documentos y búsqueda semántica
- Asambleas y votación
- Facturas y albaranes con IA; proveedores y clientes
- Área del socio (capital y aportaciones)
- Asistente de IA con citas, historial y modelo local o externo
- Marca por cooperativa (color y logo)

Trabajo futuro

- Notificaciones push
- Sentry
- E2E con Maestro
- Reclamaciones de facturas
- Binarios nativos (EAS Build)
- Invitaciones por correo

Arquitectura: monolito modular



Todo en Docker en un servidor propio · acceso público solo a la API, por Cloudflare Tunnel

Decisiones clave: 19 ADR

001	App híbrida con Expo
002	Monolito modular + Clean Architecture
003	Multi-cooperativa, BD compartida
004	Permisos por perfil, módulo y alcance
005	Ficheros S3 tras un puerto
006	Embeddings locales (bge-m3, Ollama)
007	Claude para visión (sustituido por 009)
008	Código en inglés, UI localizada
009	Gemini para visión y lenguaje
010	Embeddings Gemini (alternativa en la nube)
011	README raíz bilingüe
012	Servidor propio + Cloudflare Tunnel
013	Modelo local para el asistente
014	Modelo del asistente por cooperativa
015	qwen3:4b-instruct por defecto
016	Invitaciones por enlace del administrador
017	Dominio propio para la web
018	Color y logo por cooperativa
019	Proveedor externo por cooperativa

Seguridad y multi-cooperativa

Denegar por defecto

Cada caso de uso comprueba el permiso; la app solo oculta

Aislamiento

404 si el recurso es de otra cooperativa, 403 sin permiso; también en RAG y herramientas

Contraseñas

Argon2id, bloqueo por intentos fallidos y JWT con refresh rotatorio

Ficheros y marca

URLs firmadas; logo verificado por contenido, SVG rechazado; color con contraste AA

CORS y secretos

Orígenes permitidos; secretos fuera del repo; claves de proveedor cifradas (AES-GCM)

Tests de seguridad

Matriz perfiles x módulos y aislamiento entre cooperativas

IA en el producto

Asistente con modelo local

Ollama en el servidor propio (qwen3:4b-instruct); modelo elegible por cooperativa, o proveedor externo con su clave

RAG con citas y herramientas

pgvector filtrado por cooperativa; herramientas de solo lectura con los permisos del usuario

Gemini para visión

Extrae facturas (esquema Zod y confirmación humana) y explica discrepancias; con reintentos y alternativa

Conversación

Historial que se reabre y se elimina; respuestas en Markdown; "¿cuánto he aportado?" y proveedores/clientes

Embeddings locales con bge-m3 (Ollama) · Gemini solo para visión y explicaciones

IA en el desarrollo

119 / 120

commits co-autorados con Claude (99 %)

56 historias de usuario
19 ADR

- Spec Driven Development: especificación antes del código
- CLAUDE.md como contrato con el agente; ESLint protege las fronteras
- TDD en el dominio; IDs de requisitos en tests y commits
- Agentes en paralelo, cada uno en su git worktree
- Evaluaciones con el modelo real (23 preguntas)
- Benchmark de modelos locales antes de decidir

Calidad

941

tests unitarios en 92 ficheros

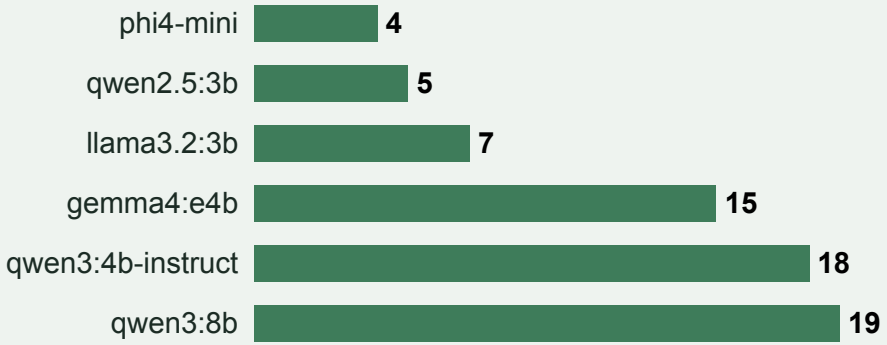
477

tests de integración en 25 ficheros

Puerta de calidad y CI

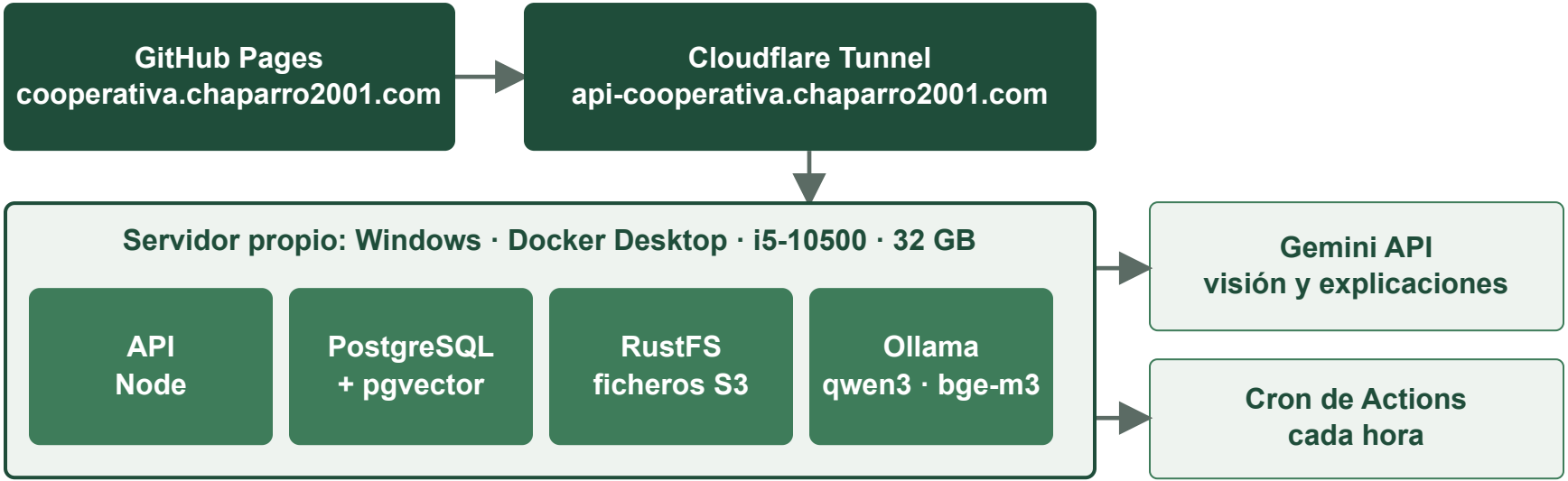
Husky en pre-commit; Actions: formato, lint, tipos, tests

Benchmark local del asistente: aciertos de 22



Mac de 12 CPU, solo CPU · el conjunto tiene hoy 23 preguntas

Despliegue gratuito, sin tarjeta



Coste: 0 EUR · Render y Railway pedían tarjeta; Neon, Supabase y Render siguen documentados

Límites: solo responde con el servidor encendido · sin copias de seguridad · no subir datos reales

Demo

URL: https://cooperativa.chaparro2001.com

Contraseña de la demo: <contraseña demo>

Perfil	Usuario	Qué se enseña
Socio	socio@sanisidro.test	Comunicados, documentos, asamblea y voto, área del socio, asistente
Gestor	gestor@sanisidro.test	Facturas, escaneo con IA, discrepancias, proveedores y clientes
Consejo	consejo@sanisidro.test	Comunicados, asambleas, vigilar pagos y cobros
Administrador	admin@sanisidro.test	Miembros e invitación, apariencia, modelo del asistente

Lecciones aprendidas y trabajo futuro

Lecciones

- Sin tarjeta no hay nube gratuita: servidor propio con Cloudflare Tunnel
- La cuota de Gemini (~20 peticiones/día en Flash) llevó al modelo local
- Latencia en CPU medida: el 4b es más rápido; el 8b, algo más preciso
- Agentes en paralelo exigen disciplina de integración
- Evaluar con el modelo real encontró errores reales
- Especificar primero, CLAUDE.md y ESLint protegen la arquitectura

Trabajo futuro

- Notificaciones push
- Sentry
- E2E con Maestro
- Reclamaciones de facturas
- Binarios nativos (EAS Build)
- Invitaciones por correo

Gracias

Repositorio: github.com/chaparro2001/multiCooperativa

Demo: cooperativa.chaparro2001.com

API: api-cooperativa.chaparro2001.com

Vídeo: <URL vídeo>

Juan Chaparro García · TFM Máster de Desarrollo con IA · BIG school · Octubre 2026